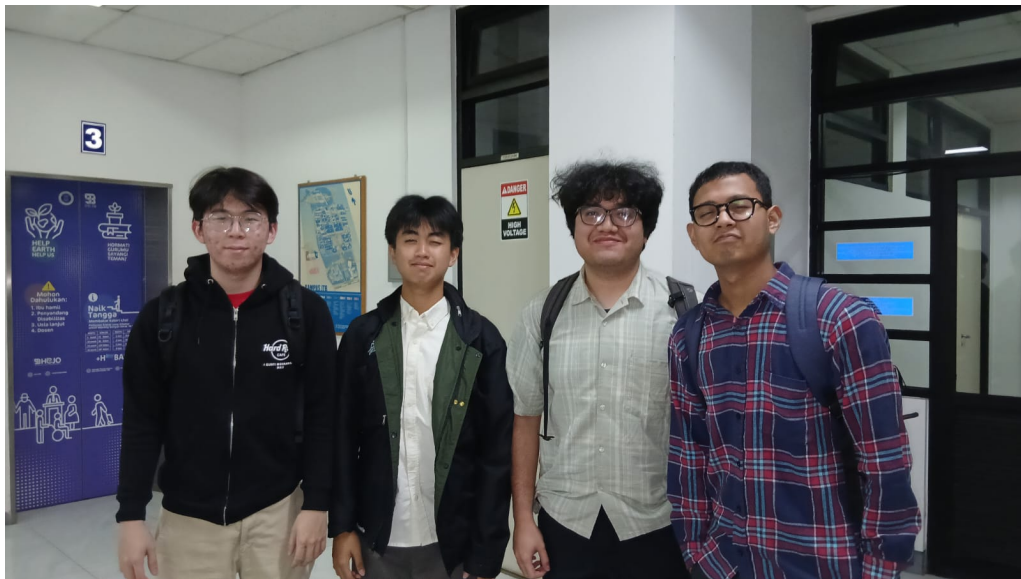


Tugas Besar

Milestone 4

IF2224 Teori Bahasa Formal dan Otomata
Intermediate Code and Interpreter
Semester 2 Tahun 2025/2026



Disusun oleh:

Kai

Jonathan Kris Wicaksono	(13524023)
Vincent Rionarlie	(13524031)
Muhammad Aufar Rizqi Kusuma	(13524061)
Bryan Pratama Putra Hendra	(13524067)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	4
1.1	Latar Belakang	4
1.2	Rumusan Masalah	4
1.3	Tujuan	5
1.4	Batasan Masalah	5
2	Dasar Teori	6
2.1	Intermediate Representation	6
2.2	Three-Address Code	7
2.3	Stack Machine dan Instruction Set Architecture	8
2.4	Activation Record dan Stack Frame	9
2.5	Translasi Control Flow dan Backpatching	10
2.6	Runtime Vulnerabilities	12
2.6.1	Kerentanan pada Stack	12
2.6.2	Kerentanan Memori dan Akses: Out-of-Bounds Variable Access	13
2.6.3	Kerentanan Alur Eksekusi: Invalid Jump Targets	13
2.6.4	Kerentanan Aritmatika: Numerical Overflow/Underflow	14
3	Perancangan dan Implementasi	16
3.1	Gambaran Umum Program	16
3.2	Struktur Direktori dan Justifikasi Pilihan Implementasi	17
3.3	Instruction Set Arion	18
3.4	Implementasi Intermediate Code Generator	20
3.4.1	Inisialisasi Memori	20
3.4.2	Translasi Statement	21
3.4.3	Translasi Ekspresi	21
3.4.4	Translasi Procedure Call	21
3.4.5	Backpatching	21
3.5	Implementasi Interpreter	21
3.5.1	Siklus Eksekusi	22
3.5.2	Manajemen Stack	22
3.5.3	Frame Management	22
3.5.4	Validasi Keamanan	22
3.6	Format Output	23



4	Pengujian	24
4.1	Strategi Pengujian	24
4.2	Detail Pengujian	25
4.2.1	Kasus 1: Operasi Dasar	25
4.2.2	Kasus 2: Control Flow Bersarang	25
4.2.3	Kasus 3: Perulangan While	26
4.2.4	Kasus 4: Perulangan For	27
4.2.5	Kasus 5: Case Statement	28
4.2.6	Kasus 6: Array Dua Dimensi	29
4.2.7	Kasus 7: Runtime Error Division by Zero	29
4.2.8	Kasus 8: Nested Procedure dengan Static Link	30
4.2.9	Kasus 9: Mutual Recursion (Keterbatasan)	31
4.3	Pengujian Error Runtime	31
4.4	Analisis Kegagalan dan Perbaikan	32
4.5	Analisis Keterbatasan Pengujian	32
5	Penutup	33
5.1	Kesimpulan	33
5.2	Saran	33
6	Lampiran	35
6.1	Tautan GitHub	35
6.2	Tabel Kontribusi	36
6.3	Cara Menjalankan Program	36



Daftar Algoritma

1	Fungsi base untuk Static Link Traversal	9
2	Validasi Sebelum Pembuatan Frame Baru	12
3	Validasi Target Jump Sebelum Eksekusi	14
4	Siklus Fetch-Decode-Execute	22

Bab 1

Pendahuluan

1.1 Latar Belakang

Setelah melewati tahap lexical analysis, syntax analysis, dan semantic analysis, compiler telah memiliki representasi program yang valid secara sintaksis maupun semantik dalam bentuk decorated AST dan symbol table. Namun, decorated AST masih berbentuk pohon yang tidak dapat dieksekusi secara langsung oleh mesin. CPU dan memori komputer dirancang untuk mengeksekusi instruksi secara sekuensial, bukan dengan melompat-lompat dalam struktur pohon yang kompleks. Oleh karena itu, diperlukan sebuah jembatan yang meratakan decorated AST menjadi daftar instruksi sederhana dan sekuensial yang disebut intermediate code.

Pada Milestone 1, Arion Compiler telah memiliki lexer yang mengubah source code menjadi token stream. Pada Milestone 2, token tersebut diproses oleh Recursive Descent Parser untuk menghasilkan parse tree. Pada Milestone 3, parse tree dikonversi menjadi AST, dianalisis secara semantik, dan diperkaya dengan anotasi tipe data, scope, serta referensi ke symbol table. Hasil akhirnya adalah decorated AST beserta tiga tabel simbol `tab`, `btab`, dan `atab`.

Milestone 4 berfokus pada dua komponen terakhir dalam pipeline compiler Arion, yaitu intermediate code generation dan interpreter. Intermediate code generator bertugas mengubah decorated AST menjadi daftar instruksi standar yang mirip dengan bahasa mesin namun masih mudah dibaca manusia. Instruksi-instruksi ini kemudian dieksekusi oleh interpreter yang berperan sebagai virtual machine. Interpreter membaca instruksi satu per satu, mengelola memori melalui stack, mengevaluasi operasi matematika, mengatur alur kontrol program, dan menghasilkan output nyata ke layar.

1.2 Rumusan Masalah

Permasalahan utama pada Milestone 4 adalah bagaimana membangun intermediate code generator dan interpreter yang dapat memanfaatkan decorated AST hasil Milestone 3 untuk menghasilkan output final dari sebuah program Arion. Permasalahan tersebut diuraikan menjadi beberapa pertanyaan sebagai berikut. Pertama, bagaimana mengonversi decorated AST menjadi intermediate code berupa daftar

instruksi sekuensial menggunakan pendekatan Three-Address Code. Kedua, bagaimana merancang instruction set yang mencakup operasi load, store, aritmatika, perbandingan, jump, call, dan return. Ketiga, bagaimana mengimplementasikan stack machine untuk mengelola memori eksekusi, termasuk stack frame, static link, dynamic link, dan return address. Keempat, bagaimana melakukan translasi control flow (`if`, `while`, `for`, `repeat`, `case`) ke dalam instruksi `JMP` dan `JPC` dengan label. Kelima, bagaimana menangani kerentanan runtime seperti stack overflow, out-of-bounds access, invalid jump target, dan numerical overflow. Keenam, bagaimana menampilkan output intermediate code dan hasil eksekusi program secara terformat.

1.3 Tujuan

Tujuan pengerjaan Milestone 4 adalah membangun tahap intermediate code generation dan interpreter pada Arion Compiler. Secara khusus, tujuan yang ingin dicapai adalah membuat intermediate code generator yang dapat mengubah decorated AST menjadi daftar instruksi `LIT`, `LOD`, `STO`, `INT`, `JMP`, `JPC`, `OPR`, `CAL`, dan `RET`. Selain itu, diimplementasikan pula stack machine dengan siklus fetch-decode-execute untuk menjalankan intermediate code, pengelolaan memori eksekusi melalui stack termasuk alokasi frame, push-pop operand, dan pengaturan stack frame untuk procedure call, penanganan translasi control flow dari struktur pohon menjadi instruksi sekuensial dengan label dan jump, penambahan deteksi dan penanganan runtime vulnerabilities pada interpreter, serta penghasilan output berupa intermediate code dan output eksekusi program sebagai hasil akhir compiler.

1.4 Batasan Masalah

Implementasi Milestone 4 dibatasi pada backend compiler Arion. Program tidak lagi melakukan analisis sintaksis atau semantik, karena validitas program diasumsikan sudah diperiksa oleh tahap sebelumnya. Intermediate code generator bekerja pada decorated AST yang dibangun dari parse tree hasil parser, sedangkan interpreter mengeksekusi intermediate code tanpa mempedulikan sintaks bahasa sumber.

Pengecekan runtime yang diimplementasikan berfokus pada stack overflow, stack underflow, stack smashing, out-of-bounds array access, invalid jump target, dan numerical overflow atau underflow. Fitur yang belum didukung pada tahap ini meliputi function call dalam ekspresi (`FuncCall`), record field access dalam intermediate code, serta boolean operator `and`, `or`, dan `not` pada level code generation.

Bab 2

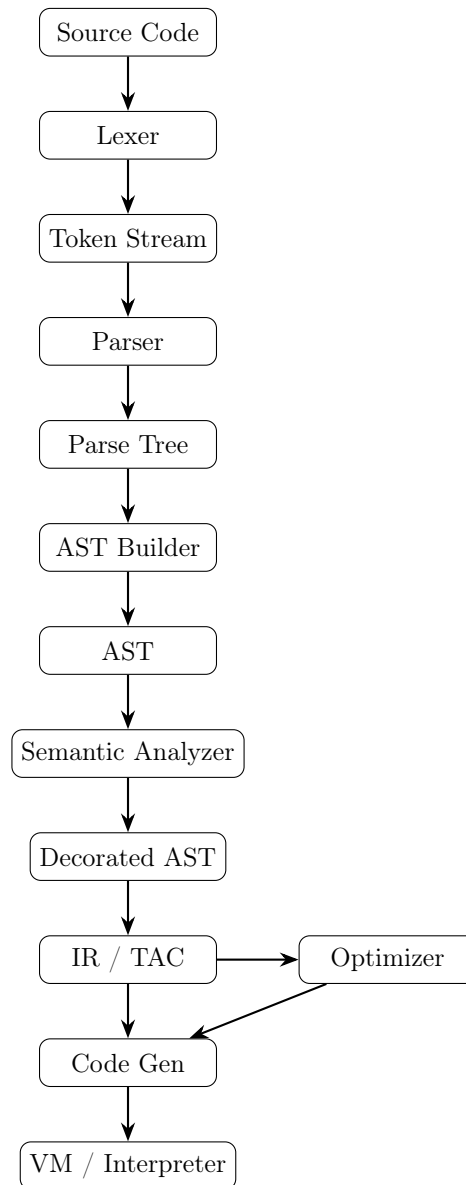
Dasar Teori

2.1 Intermediate Representation

Intermediate representation (IR) adalah representasi internal program yang berada di antara bahasa sumber dan kode mesin target. Secara formal, IR dapat didefinisikan sebagai struktur data atau bahasa perantara yang menyimpan semantik program secara lengkap namun independen dari sintaks bahasa sumber maupun arsitektur mesin target [1, 3]. Keberadaan IR memungkinkan pemisahan yang jelas antara frontend dan backend compiler.

Dalam taksonomi compiler, IR diklasifikasikan berdasarkan kedekatannya dengan sumber atau mesin. High-level IR mempertahankan struktur kontrol dan tipe data tingkat tinggi sehingga cocok untuk optimasi berbasis sumber, seperti eliminasi subekspresi umum. Low-level IR mendekati bahasa mesin dengan register tak terbatas atau memori eksplisit sehingga memudahkan alokasi register dan penjadwalan instruksi. Beberapa bentuk IR yang dikenal dalam literatur adalah Three-Address Code (TAC), Static Single Assignment (SSA), bytecode (misalnya JVM), dan P-code yang dikembangkan oleh Wirth untuk mesin stack [2, ?].

Ada tiga alasan fundamental mengapa compiler modern menggunakan IR. Pertama, *machine independence*: frontend compiler tidak perlu mengetahui arsitektur target karena hasil analisis disimpan dalam IR. Kedua, *optimization opportunity*: IR menyediakan level abstraksi yang lebih mudah dianalisis dan dioptimasi dibandingkan source code atau kode mesin. Ketiga, *portability*: dengan IR yang sama, satu frontend dapat dipasangkan dengan banyak backend untuk mesin berbeda, dan sebaliknya satu backend dapat melayani banyak bahasa sumber.



Gambar 2.1: Pipeline compiler lengkap dari source code hingga eksekusi mesin virtual

Gambar 2.1 menunjukkan pipeline compiler lengkap mulai dari source code hingga eksekusi oleh mesin virtual. Frontend meliputi lexer, parser, AST builder, dan semantic analyzer yang menghasilkan decorated AST. Backend dimulai dari intermediate code generator yang meratakan decorated AST menjadi IR berupa TAC. IR tersebut dapat dioptimasi sebelum diterjemahkan ke kode mesin target oleh code generator, atau dieksekusi langsung oleh virtual machine interpreter.

2.2 Three-Address Code

Three-Address Code (TAC) adalah format IR yang setiap instruksinya melibatkan maksimal tiga alamat: dua operand sumber dan satu lokasi tujuan [1]. Dalam konteks TAC, *address* tidak selalu berupa alamat memori fisik, melainkan dapat

berupa nama variabel, konstanta literal, atau variabel sementara yang dihasilkan compiler [3]. Variabel sementara ini biasanya dinamai t_1 , t_2 , t_3 , dan seterusnya, sesuai dengan urutan pembuatannya selama traversal ekspresi.

Pembentukan TAC dari ekspresi kompleks dilakukan melalui traversal *post-order* pada pohon ekspresi. Setiap node internal diterjemahkan menjadi satu instruksi TAC, sedangkan daun-daunnya menjadi operand. Sebagai contoh, ekspresi $x := (a + b) * (c - d)$ menghasilkan empat instruksi TAC berikut.

Contoh TAC

```
t1 := a + b
t2 := c - d
t3 := t1 * t2
x  := t3
```

Nama variabel sementara umumnya mengikuti konvensi t yang diikuti nomor urut. Compiler dapat menggunakan berbagai strategi penamaan, namun yang paling penting adalah keunikan nama dalam satu prosedur dan tidak bertabrakan dengan nama variabel program. Setelah nilai variabel sementara tidak lagi dibutuhkan, nama tersebut dapat *reuse*. Analisis *liveness* digunakan untuk menentukan kapan variabel sementara masih *hidup* (masih akan digunakan di masa depan) dan kapan sudah *mati* (tidak akan digunakan lagi). Variabel yang sudah mati dapat di-assign ulang ke ekspresi baru tanpa kehilangan informasi.

Dalam literatur compiler, TAC direpresentasikan dalam tiga bentuk utama: *quadruples*, *triples*, dan *indirect triples* [1]. Quadruples menyimpan setiap instruksi sebagai tupel empat elemen (op, arg1, arg2, result), sehingga setiap instruksi dapat diacu secara eksplisit. Triples menyimpan tiga elemen (op, arg1, arg2) dengan hasil implisit berupa indeks instruksi itu sendiri, yang mengurangi ruang penyimpanan namun mempersulit pemindahan kode. Indirect triples menambahkan tabel pointer ke triples, sehingga memungkinkan penyusunan ulang instruksi tanpa mengubah isi triples. Tabel 2.1 membandingkan ketiga representasi untuk ekspresi $x = a + b * c$.

Tabel 2.1: Perbandingan representasi TAC

No	Quadruples	Triples	Indirect Triples
0	(MUL, b, c, t1)	(0) MUL b c	(0) $\rightarrow$ 100
1	(ADD, a, t1, t2)	(1) ADD a (0)	(1) $\rightarrow$ 101
2	(ASSIGN, t2, _, x)	(2) ASSIGN x (1)	(2) $\rightarrow$ 102

2.3 Stack Machine dan Instruction Set Architecture

Arsitektur mesin eksekusi untuk intermediate code dapat diklasifikasikan menjadi tiga paradigma utama: *stack machine*, *register machine*, dan *accumulator machine*. Stack machine menggunakan struktur stack implisit sebagai tempat operan; setiap instruksi mempop operand dari stack, melakukan operasi, dan mempush hasilnya kembali [?]. Register machine menggunakan register eksplisit sebagai operand (misalnya

x86, ARM), sedangkan accumulator machine hanya memiliki satu register implisit sebagai akumulator. Stack machine dipilih secara luas pada compiler edukasional karena format instruksinya sangat sederhana dan codegen dari AST menjadi sangat langsung.

Salah satu contoh klasik stack machine adalah P-code machine yang dikembangkan Niklaus Wirth untuk compiler Pascal. P-code menggunakan instruksi berbasis stack dengan format tiga field: opcode, level leksikal, dan operand [?]. Level leksikal merepresentasikan kedalaman *static scoping*, sedangkan operand menyimpan nilai literal, offset memori, atau alamat instruksi. Mekanisme ini memungkinkan akses variabel non-lokal melalui traversal *static link chain*.

Konsep *lexical addressing* menjadi fondasi akses variabel dalam bahasa dengan *static scoping*. Setiap variabel diakses melalui pasangan (`level`, `offset`), di mana `level` menunjukkan jarak leksikal dari variabel terhadap scope saat ini, dan `offset` menunjukkan posisi relatif dalam frame memori. Static scoping berbeda dengan dynamic scoping: pada static scoping, ruang lingkup variabel ditentukan oleh struktur tekstual program (letak deklarasi), sedangkan pada dynamic scoping ditentukan oleh urutan pemanggilan fungsi pada runtime. Compiler modern hampir selalu menggunakan static scoping karena lebih mudah dianalisis dan diprediksi.

Fungsi standar untuk menelusuri static link dalam interpreter stack machine adalah `base(1, b)`, yang secara rekursif mengikuti static link sebanyak `l` level dari base pointer `b` saat ini. Fungsi ini didefinisikan sebagai berikut.

Algoritma 1: Fungsi base untuk Static Link Traversal

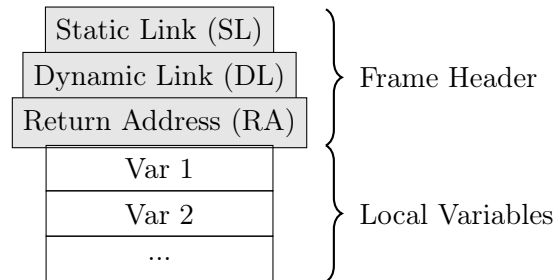
```
1: function BASE( $l, b$ )
2:   while  $l > 0$  do
3:      $b \leftarrow \text{stack}[b]$                                 ▷ static link ada di slot 0
4:      $l \leftarrow l - 1$ 
5:   end while
6:   return  $b$ 
7: end function
```

Pada saat eksekusi, instruksi `LOD 1 a` (load) dan `STO 1 a` (store) menggunakan fungsi `base(1, b)` untuk menemukan base pointer frame yang tepat sebelum mengakses slot offset `a`. Jika `l = 0`, variabel berada pada frame saat ini sehingga tidak perlu traversal. Jika `l > 0`, interpreter mengikuti static link chain sebanyak `l` level, yang memungkinkan prosedur bersarang untuk mengakses variabel dari scope induk secara leksikal.

2.4 Activation Record dan Stack Frame

Activation record, atau *stack frame*, adalah blok memori kontigu yang dibuat setiap kali sebuah prosedur dipanggil. Blok ini menyimpan seluruh konteks eksekusi prosedur tersebut, termasuk variabel lokal, argumen, alamat pengembalian, serta tautan ke frame lain [1]. Stack frame bekerja dengan prinsip LIFO (Last In, First Out): frame baru di-push ke pucuk stack saat pemanggilan, dan di-pop saat prosedur selesai, sehingga seluruh variabel lokalnya otomatis musnah.

Setiap stack frame memiliki tiga slot header wajib. **Static link** (slot 0) menunjuk ke base pointer frame induk secara leksikal berdasarkan kedalaman *static scoping*. **Dynamic link** (slot 1) menunjuk ke base pointer frame pemanggil pada runtime. **Return address** (slot 2) menyimpan indeks instruksi yang harus dieksekusi setelah prosedur selesai. Slot-slot setelah header digunakan untuk parameter, variabel lokal, dan operand sementara selama evaluasi ekspresi.



Gambar 2.2: Ilustrasi stack frame dengan static link, dynamic link, dan return address

Pemilihan stack sebagai struktur memori eksekusi didasarkan pada kebutuhan scope dan pemanggilan prosedur. Array biasa tidak dapat secara otomatis mengelola siklus hidup variabel lokal karena tidak memiliki mekanisme push-pop yang terkait dengan aktivasi dan deaktivasi prosedur.

2.5 Translasi Control Flow dan Backpatching

Penerjemahan struktur kontrol seperti percabangan dan perulangan ke dalam IR memerlukan teknik *syntax-directed translation* (SDT). SDT adalah metode code generation yang menggunakan atribut pada node pohon sintaks untuk membimbing pembentukan instruksi [1]. Atribut *synthesized* mengalir dari anak ke induk (misalnya fragmen kode yang dihasilkan oleh subekspresi), sedangkan atribut *inherited* mengalir dari induk ke anak (misalnya label berikutnya yang harus dituju setelah suatu blok).

Tantangan utama dalam translasi control flow adalah penanganan instruksi lompatan yang belum mengetahui targetnya pada saat instruksi tersebut dihasilkan. Solusinya adalah teknik *backpatching*. Secara formal, backpatching bekerja dengan memelihara *backpatch list*, yaitu himpunan indeks instruksi yang memiliki target lompatan kosong. Tiga operasi fundamental didefinisikan: `backpatch(L, i)` mengisi semua indeks dalam list L dengan target i ; `merge(L1, L2)` menggabungkan dua backpatch list; dan `makelist(i)` membuat list baru berisi indeks tunggal i [1].

Sebagai contoh, translasi `if-then-else` menggunakan dua backpatch list. List `false`s menyimpan indeks JPC yang melompat ke blok `else`, sedangkan list `next`s menyimpan indeks JMP yang melompat ke akhir percabangan. Setelah posisi label `else` diketahui, `false`s di-backpatch. Setelah posisi label akhir diketahui, `next`s di-backpatch.

Contoh TAC untuk IF-ELSE dengan Backpatching

```
{ evaluasi kondisi }
JPC 0 _          // falses = makelist(ip)
{ blok then }
JMP 0 _          // nexts = makelist(ip)
L_ELSE:
{ blok else }
L_END:
backpatch(falses, L_ELSE)
backpatch(nexts, L_END)
```

Perulangan **while** menggunakan pola yang sama. Label **L_START** dipasang sebelum evaluasi kondisi. Instruksi **JPC** keluar ke **L_END** yang belum diketahui, sehingga indeksnya masuk ke backpatch list. Setelah body selesai, **JMP** kembali ke **L_START**. Label **L_END** dipasang setelah body, lalu backpatch list diisi.

Contoh TAC untuk WHILE

```
L_START:
{ evaluasi kondisi }
JPC 0 _          // exits = makelist(ip)
{ blok body }
JMP 0 L_START
L_END:
backpatch(exits, L_END)
```

Perulangan **repeat-until** memiliki karakteristik berbeda karena body selalu dieksekusi setidaknya sekali. Body diterjemahkan terlebih dahulu, kemudian kondisi dievaluasi. Jika kondisi bernilai salah (belum terpenuhi), **JPC** melompat kembali ke awal body menggunakan backpatch list.

Contoh TAC untuk REPEAT-UNTIL

```
L_START:
{ blok body }
{ evaluasi kondisi }
JPC 0 _          // repeats = makelist(ip)
L_END:
backpatch(repeats, L_START)
```

Translasi **for** dan **case** juga bergantung pada backpatching. **For** loop memerlukan backpatching untuk lompatan keluar dari loop, sedangkan **case** statement memerlukan backpatching untuk setiap branch agar tidak jatuh secara implisit ke branch berikutnya (*fall-through*).

2.6 Runtime Vulnerabilities

Interpreter yang mengeksekusi IR beroperasi pada level yang lebih dekat dengan mesin dibandingkan bahasa sumber, sehingga rentan terhadap kerentanan yang berkaitan dengan manajemen memori dan alur eksekusi. *Memory safety* dalam konteks stack machine mencakup dua aspek: *spatial safety* (memastikan setiap akses memori berada dalam batas yang valid) dan *temporal safety* (memastikan memori tidak diakses setelah dilepaskan) [1].

2.6.1 Kerentanan pada Stack

Stack overflow terjadi ketika kedalaman pemanggilan prosedur melampaui kapasitas maksimum stack yang dialokasikan. Kondisi ini paling umum disebabkan oleh rekursi tak terbatas, di mana sebuah fungsi terus memanggil dirinya sendiri tanpa kondisi berhenti. Dampaknya adalah interpreter kehabisan memori untuk menyimpan frame baru, yang pada akhirnya menyebabkan program crash atau hang. Sebagai contoh, prosedur `recur` yang memanggil dirinya sendiri secara langsung akan menghasilkan error `stack overflow` setelah kedalaman frame melebihi batas yang ditetapkan.

Stack underflow merupakan kondisi inverse, yaitu ketika interpreter diinstruksikan untuk mengambil nilai dari stack operand yang sudah sepenuhnya kosong. Hal ini biasanya menandakan adanya cacat logika pada intermediate code generator, misalnya ketika sebuah operasi biner hanya memiliki satu operand di stack. Dampaknya adalah interpreter membaca nilai sampah atau memori yang tidak valid, yang dapat merusak seluruh komputasi selanjutnya.

Stack smashing adalah bentuk klasik dari buffer overflow pada stack machine. Dalam arsitektur ini, frame header (static link, dynamic link, dan return address) diletakkan secara bersebelahan dengan variabel lokal. Jika interpreter mengizinkan penulisan data melebihi ukuran variabel lokalnya, data tersebut dapat meluber dan menimpa return address. Akibatnya, saat prosedur selesai dieksekusi, interpreter akan melompat ke alamat memori yang salah, yang berpotensi mengeksekusi kode berbahaya atau menyebabkan crash sistem.

Pendekatan umum untuk menangani ketiga kerentanan stack adalah dengan menambahkan validasi sebelum setiap operasi yang berpotensi berbahaya. Sebelum membuat frame baru melalui instruksi `CAL`, interpreter harus memeriksa apakah kedalaman stack saat ini masih di bawah batas maksimum. Sebelum melakukan operasi `pop`, interpreter harus memastikan bahwa stack pointer masih berada di atas base pointer frame saat ini ditambah ukuran header. Sebelum menulis ke memori melalui `STO`, interpreter harus memverifikasi bahwa slot target bukan merupakan slot header yang dilindungi. Algoritma berikut mengilustrasikan validasi yang dilakukan sebelum pembuatan frame baru.

Algoritma 2: Validasi Sebelum Pembuatan Frame Baru

```
1: procedure VALIDATEPUSHFRAME
2:   if currentDepth  $\geq$  kMaxStackFrames then
3:     error stack overflow
4:   end if
```

5: end procedure

2.6.2 Kerentanan Memori dan Akses: Out-of-Bounds Variable Access

Out-of-bounds variable access adalah kerentanan yang terjadi ketika program mencoba mengakses indeks array di luar batas yang dideklarasikan. Dalam konteks stack machine, setiap elemen array dialokasikan secara kontigu di dalam stack frame. Jika interpreter mengizinkan akses indeks melebihi batas atas atau kurang dari batas bawah, maka akses tersebut akan mengarah ke memori milik variabel lain atau bahkan frame header, yang melanggar *spatial safety*.

Dampak dari kerentanan ini dapat sangat serius. Penulisan ke indeks yang salah dapat merusak nilai variabel lain secara diam-diam, yang menyebabkan bug yang sulit dilacak. Dalam kasus ekstrem, penulisan tersebut dapat menimpa return address dan memicu stack smashing. Sebagai contoh, jika sebuah array dideklarasikan dengan batas `array[1..3]`, namun program mencoba mengakses `array[5]`, maka interpreter harus mencegah akses tersebut sebelum mencapai memori fisik.

Pendekatan umum untuk menangani kerentanan ini adalah dengan melakukan *bounds checking* pada setiap operasi akses array. Interpreter membandingkan indeks yang diminta dengan batas bawah (*low*) dan batas atas (*high*) yang tersimpan dalam metadata array. Jika indeks berada di luar rentang tersebut, interpreter menghentikan eksekusi dan melaporkan `IndexOutOfBoundsException`. Metadata array pada Arion disimpan dalam operand instruksi `ALOD` dan `ASTO`, yang berisi base offset, jumlah dimensi, batas rendah, dan batas tinggi untuk setiap dimensi.

Contoh Bounds Checking pada Array Access

```
if index < low || index > high then
    error: IndexOutOfBoundsException
else
    address = base + (index - low)
    access memory at address
end
```

2.6.3 Kerentanan Alur Eksekusi: Invalid Jump Targets

Invalid jump targets terjadi ketika intermediate code generator menghasilkan instruksi lompatan (`JMP`, `JPC`, atau `CAL`) yang menuju ke alamat instruksi yang tidak pernah ada dalam daftar program. Kondisi ini dapat disebabkan oleh kesalahan pada mekanisme backpatching, di mana alamat target tidak diisi dengan benar setelah seluruh instruksi dihasilkan. Dampaknya adalah instruction pointer (IP) akan melompat ke luar batas vektor instruksi, yang menyebabkan interpreter membaca memori yang tidak valid dan crash secara tiba-tiba.

Sebagai contoh, jika sebuah program memiliki 20 instruksi (indeks 0 hingga 19), namun instruksi ke-5 berisi `JMP 0 25`, maka lompatan tersebut akan mengarah ke

alamat yang tidak pernah didefinisikan. Tanpa validasi, interpreter akan mencoba membaca instruksi ke-25 dan menghasilkan perilaku tak terdefinisi.

Pendekatan umum untuk menangani kerentanan ini adalah dengan melakukan *pre-execution validation* terhadap seluruh target lompatan sebelum siklus fetch-decode-execute dimulai. Interpreter melakukan satu kali traversal pada seluruh daftar instruksi untuk mengumpulkan semua alamat target yang direferensi oleh instruksi lompatan. Kemudian, setiap target tersebut diverifikasi agar berada dalam rentang valid (0 hingga `program.size() - 1`). Jika ada target yang tidak valid, interpreter langsung menghentikan eksekusi dan melaporkan `InvalidJumpTargetException` tanpa mengeksekusi satu instruksi pun. Strategi ini lebih aman karena mencegah state mesin menjadi korup sejak awal.

Algoritma 3: Validasi Target Jump Sebelum Eksekusi

```
1: procedure VALIDATEJUMPTARGETS(program)
2:   for each inst in program do
3:     if inst.opcode in {JMP, JPC, CAL} then
4:       if inst.operand < 0 or inst.operand ≥ program.size() then
5:         error InvalidJumpTargetException
6:       end if
7:     end if
8:   end for
9: end procedure
```

2.6.4 Kerentanan Aritmatika: Numerical Overflow/Underflow

Numerical overflow terjadi ketika hasil operasi aritmatika melebihi nilai maksimum yang dapat ditampung oleh tipe data integer, sedangkan **numerical underflow** terjadi ketika hasil operasi jatuh di bawah nilai minimum. Dalam representasi signed integer 32-bit, nilai maksimum adalah sekitar 2.147.483.647 dan nilai minimum adalah sekitar -2.147.483.648. Jika sebuah operasi penjumlahan menghasilkan nilai di luar rentang tersebut, maka akan terjadi *wrap-around*, di mana nilai positif besar tiba-tiba menjadi negatif, atau sebaliknya.

Dampak dari kerentanan ini dapat sangat merusak logika program. Sebagai contoh, sebuah perulangan yang menggunakan variabel penghitung integer dapat menjadi infinite loop jika variabel tersebut overflow dari nilai positif maksimum kembali ke nilai negatif. Dalam aplikasi finansial atau ilmiah, hasil perhitungan yang salah karena overflow dapat menghasilkan keputusan yang fatal.

Pendekatan umum untuk menangani kerentanan ini adalah dengan menggunakan *checked arithmetic*. Sebelum melakukan operasi, interpreter memeriksa apakah hasil operasi akan melampaui batas tipe data. Untuk penjumlahan dua bilangan positif, interpreter memeriksa apakah $a > \text{INT_MAX} - b$. Untuk pengurangan, interpreter memeriksa apakah $a < \text{INT_MIN} + b$. Untuk perkalian, pemeriksaan lebih kompleks karena harus mempertimbangkan tanda kedua operand. Jika pemeriksaan menunjukkan bahwa overflow atau underflow akan terjadi, interpreter menghentikan

eksekusi dan melaporkan `OverflowError` atau `UnderflowError`. Strategi ini lebih aman dibandingkan *silent wrap-around* karena memberi tahu programmer bahwa ada kesalahan logika yang perlu diperbaiki.

Contoh Checked Addition

```
function checkedAdd(a, b):  
    if a > 0 and b > INT_MAX - a then  
        error: OverflowError  
    else if a < 0 and b < INT_MIN - a then  
        error: UnderflowError  
    else  
        return a + b  
    end
```

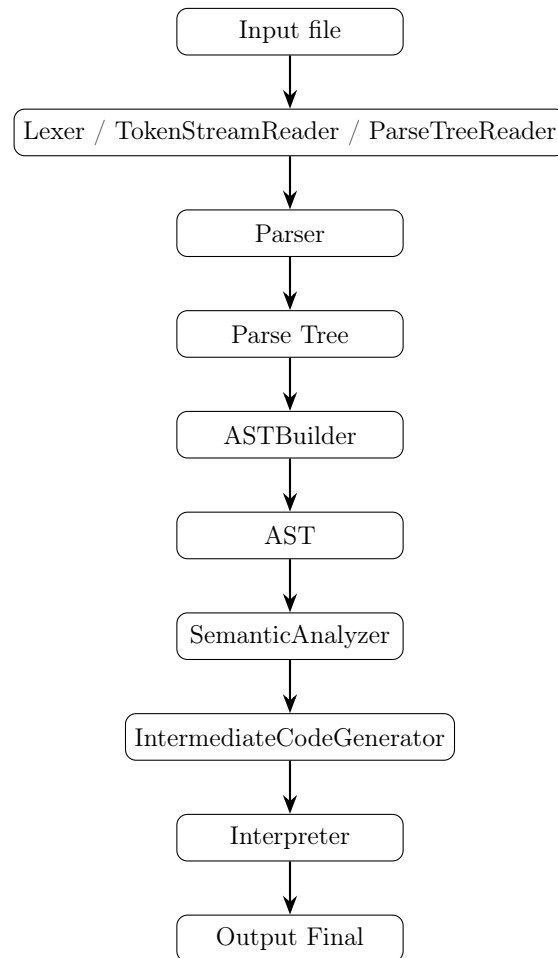

Bab 3

Perancangan dan Implementasi

3.1 Gambaran Umum Program

Implementasi Milestone 4 melanjutkan pipeline Milestone 1, Milestone 2, dan Milestone 3. Alur utama berada pada `src/main.cpp`. Program membaca file input, lalu menentukan bentuk input tersebut. Jika input adalah parse tree, program menggunakan `ParseTreeReader`. Jika input adalah token stream, program menggunakan `TokenStreamReader` lalu menjalankan parser. Jika input adalah source code Arion, program menjalankan lexer dan parser.

Setelah parse tree tersedia, program menjalankan `ASTBuilder` untuk membentuk AST. AST tersebut kemudian dianalisis oleh `SemanticAnalyzer`. Jika tidak ada error, program menjalankan `IntermediateCodeGenerator` untuk mengubah decorated AST menjadi daftar instruksi intermediate code. Terakhir, `Interpreter` mengeksekusi intermediate code tersebut dan menghasilkan output final.



Gambar 3.1: Pipeline Program Milestone 4

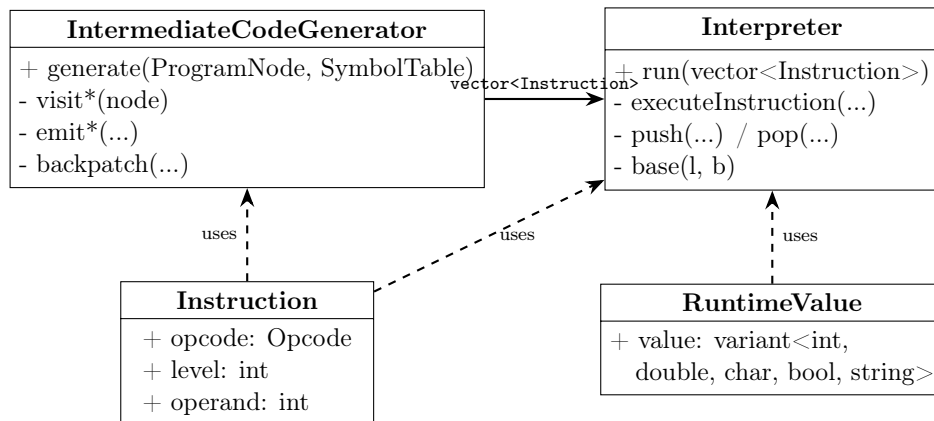
3.2 Struktur Direktori dan Justifikasi Pilihan Implementasi

Bagian Milestone 4 terutama berada pada direktori `src/codegen/` dan `src/interpreter/`. Berkas utama yang digunakan adalah `intermediate_code.h` dan `intermediate_code.cpp` yang berisi definisi instruction set, intermediate code generator, dan konversi decorated AST menjadi daftar instruksi. Berkas `interpreter.h` dan `interpreter.cpp` mengimplementasikan virtual machine dengan siklus fetch-decode-execute, stack management, dan eksekusi instruksi. Berkas `runtime_value.h` dan `runtime_value.cpp` menyediakan representasi nilai runtime.

Pemilihan struktur data `std::vector` untuk stack didasarkan pada tiga pertimbangan. Pertama, `std::vector` menyediakan *amortized constant time* untuk operasi `push_back`, yang sebanding dengan kompleksitas operasi push pada stack. Kedua, memori yang dialokasikan bersifat *contiguous*, sehingga memanfaatkan *cache locality* pada modern CPU dan mempercepat akses berurutan. Ketiga, `std::vector` mendukung pertumbuhan dinamis sehingga interpreter tidak perlu mengalokasikan memori besar secara statis di awal eksekusi, melainkan dapat memperluas stack sesuai kebutuhan runtime.

Pemilihan `std::variant` untuk `RuntimeValue` didasarkan pada kebutuhan type safety tanpa overhead manual tag management. Berbeda dengan *union* tradisional dalam C, `std::variant` secara otomatis mengelola konstruksi dan destruksi objek serta menyediakan *compile-time type checking* melalui `std::visit`. Hal ini mencegah bug *type punning* yang umum terjadi pada union manual, sekaligus mendukung tipe heterogen (`int`, `double`, `char`, `bool`, `std::string`) dalam satu struktur data yang aman.

Antarmuka antara `IntermediateCodeGenerator` dan `Interpreter` didefinisikan melalui struktur data `Instruction` yang berisi tiga field: `opcode` (kode operasi), `level` (level leksikal untuk akses variabel non-lokal), dan `operand` (nilai literal, offset memori, atau alamat instruksi). Generator menghasilkan `std::vector<Instruction>` yang sepenuhnya terpisah dari logika eksekusi interpreter. Desain ini memastikan loose coupling: perubahan algoritma codegen tidak mempengaruhi interpreter selama format instruksi tetap sama, dan sebaliknya perubahan strategi eksekusi tidak mempengaruhi cara instruksi dihasilkan.



Gambar 3.2: Diagram ketergantungan modul utama Milestone 4

3.3 Instruction Set Arion

Instruction set yang diimplementasikan pada Arion Compiler terdiri dari sembilan instruksi utama. Setiap instruksi memiliki opcode, level, dan operand. Level merepresentasikan lexical level untuk akses variabel non-lokal, sedangkan operand menyimpan nilai literal, alamat memori, atau nomor operasi.

Tabel 3.1: Daftar instruksi intermediate code Arion

Instruksi	Operasi	Keterangan
LIT v	Load Literal	Memasukkan nilai literal v ke dalam stack.
LOD a	Load Value	Memuat nilai dari alamat a.
STO a	Store Value	Menyimpan nilai ke alamat a.
CAL l	Call	Memanggil prosedur di alamat l.
INT m	Initiate Memory	Mengalokasikan atau dealokasikan m slot stack.
JMP l	Unconditional Jump	Lompat ke alamat l tanpa kondisi.
JPC l	Conditional Jump	Lompat ke alamat l bila kondisi tidak memenuhi.
OPR o	Operation	Memanggil operasi o (aritmatika, logika, I/O).
RET	Return	Keluar dari prosedur dan kembali ke pemanggil.

Instruksi OPR memiliki sub-operasi yang lebih spesifik. Sub-operasi yang didukung meliputi negasi, penjumlahan, pengurangan, perkalian, pembagian, modulo, perbandingan (equal, not equal, less, greater, less-equal, greater-equal), serta operasi I/O WRT dan WRTLN.

Tabel 3.2: Daftar sub-operasi instruksi OPR

No	Operation Name	Function
1	NEG	Negasi nilai teratas dari stack.
2	ADD	Menambah dua nilai teratas dari stack.
3	SUB	Mengurangi dua nilai teratas dari stack.
4	MUL	Mengali dua nilai teratas dari stack.
5	DIV	Membagi dua nilai teratas dari stack.
6	MOD	Modulus dua nilai teratas dari stack.
7	EQL	Memeriksa kesamaan dua nilai teratas.
8	NEQ	Memeriksa ketidaksamaan dua nilai teratas.
9	LSS	Memeriksa apakah nilai pertama kurang dari kedua.
10	GEQ	Memeriksa apakah nilai pertama lebih dari sama dengan kedua.
11	GTR	Memeriksa apakah nilai pertama lebih dari kedua.
12	LEQ	Memeriksa apakah nilai pertama kurang dari sama dengan kedua.
13	WRT	Menulis output tanpa newline.
14	WRTLN	Menulis output dengan newline.

3.4 Implementasi Intermediate Code Generator

Intermediate code generator diimplementasikan pada class `CodeGen::IntermediateCodeGenerator`. Fungsi utama `generate` menerima `ProgramNode` dari decorated AST dan `SymbolTable` hasil semantic analysis, kemudian menghasilkan `Result` yang berisi daftar instruksi dan error (jika ada).

Generator bekerja secara top-down, mengunjungi setiap node AST dan mengeluarkan instruksi yang relevan. Detail implementasi meliputi beberapa aspek berikut.

3.4.1 Inisialisasi Memori

Fungsi `initialFrameSize` menghitung ukuran frame awal berdasarkan informasi `btab` dari program. Frame size meliputi `kFrameHeaderSize` (3 slot untuk static link, dynamic link, dan return address) ditambah jumlah parameter (`psze`) dan variabel lokal (`vsze`). Instruksi INT pertama yang dihasilkan adalah `INT 0 frameSize`.

3.4.2 Translasi Statement

Setiap jenis statement memiliki fungsi `visit` sendiri. Untuk **Assignment**, generator mengevaluasi ekspresi value terlebih dahulu, lalu memanggil `emitStore` untuk target. Target dapat berupa variabel sederhana atau array access. Untuk **If**, kondisi dievaluasi, diikuti `JPC` ke label `else` atau `end`. Blok `then` diterjemahkan, lalu `JMP` ke `end`. Jika ada `else`, blok `else` diterjemahkan, lalu label `end` dipasang. Untuk **While**, label loop start dipasang, kondisi dievaluasi, `JPC` ke `end`, body diterjemahkan, `JMP` kembali ke start, lalu label `end` dipasang. Untuk **Repeat**, label loop start dipasang, body diterjemahkan, kondisi dievaluasi, `JPC` kembali ke start. Untuk **For**, variabel loop diinisialisasi dengan start expression. Label loop start dipasang, variabel loop dan end expression dievaluasi, dibandingkan dengan `LEQ` atau `GEQ`, `JPC` ke `end`, body diterjemahkan, variabel loop diincrement atau decrement, `JMP` kembali ke start, lalu label `end` dipasang. Untuk **Case**, pada setiap branch selector dan label dievaluasi, dibandingkan dengan `EQL`, `JPC` ke label berikutnya jika tidak cocok. Jika cocok, eksekusi jatuh ke body branch. Setelah body, `JMP` ke `end case`. Semua `JMP` ke `end` di-backpatch setelah seluruh branch diproses.

3.4.3 Translasi Ekspresi

Ekspresi diterjemahkan secara rekursif. Literal integer, real, char, string, dan Boolean menghasilkan `LIT`. Variabel menghasilkan `LOD`. Array access dengan satu atau dua dimensi menghasilkan `ALOD` dengan operand yang menyimpan metadata dimensi. Operator biner menghasilkan evaluasi kedua operand diikuti `OPR`. Unary minus menghasilkan evaluasi operand diikuti `OPR NEG`.

3.4.4 Translasi Procedure Call

Saat memanggil prosedur, generator mengevaluasi argumen (push ke stack), lalu mengeluarkan `CAL` dengan target alamat prosedur. Jika alamat belum diketahui saat pemanggilan, instruksi `CAL` menggunakan placeholder yang di-backpatch setelah seluruh deklarasi prosedur diproses. Setelah `CAL` selesai, generator mengeluarkan `INT` dengan nilai negatif untuk membersihkan argumen dari stack caller.

3.4.5 Backpatching

Beberapa instruksi seperti `JMP`, `JPC`, dan `CAL` memerlukan backpatching karena alamat tujuan belum diketahui saat instruksi pertama kali dihasilkan. Generator menyimpan indeks instruksi yang perlu di-backpatch dalam variabel sementara, lalu mengisi operandnya setelah alamat tujuan diketahui.

3.5 Implementasi Interpreter

Interpreter diimplementasikan pada class `Interpreter::Interpreter`. Fungsi utama `run` menerima daftar instruksi, menginisialisasi stack dan register, lalu mengeksekusi instruksi satu per satu dalam siklus fetch-decode-execute.

3.5.1 Siklus Eksekusi

Interpreter menggunakan variabel `ip` (instruction pointer), `bp` (base pointer), dan `sp` (stack pointer). Siklus berjalan selama `ip` masih dalam rentang valid dan tidak ada error.

Algoritma 4: Siklus Fetch-Decode-Execute

```
1: procedure RUN(program)
2:   reset()
3:   validateJumpTargets(program)
4:   while not halted and ip valid do
5:     inst = program[ip]
6:     advance = true
7:     if not EXECUTEINSTRUCTION(inst, program, advance) then
8:       return error
9:     end if
10:    if advance then
11:      ip += 1
12:    end if
13:  end while
14:  return result
15: end procedure
```

3.5.2 Manajemen Stack

Stack diimplementasikan menggunakan `std::vector<RuntimeValue>`. Fungsi `push` menambahkan nilai ke stack dengan pengecekan batas maksimal `kMaxStackSlots`. Fungsi `pop` mengambil nilai dari stack dengan pengecekan underflow. Fungsi `ensureStackSize` mengubah ukuran vector sesuai kebutuhan.

Untuk instruksi `INT` dengan nilai negatif, interpreter menyusutkan stack dengan menghapus elemen dari belakang, asalkan tidak menyusut di bawah frame header saat ini.

3.5.3 Frame Management

Instruksi `CAL` membuat frame baru dengan menyimpan static link, dynamic link, dan return address. Static link dihitung berdasarkan lexical level delta menggunakan fungsi `base(1, b)` yang menelusuri static link chain sebanyak `l` level. Dynamic link adalah `bp` saat ini. Return address adalah `ip + 1`. Instruksi `RET` memulihkan `ip` dari return address, `bp` dari dynamic link, dan menghapus frame saat ini.

3.5.4 Validasi Keamanan

Interpreter memvalidasi target jump sebelum eksekusi dimulai untuk memastikan semua `JMP`, `JPC`, dan `CAL` menunjuk ke alamat yang valid. Selama eksekusi, interpreter

juga memvalidasi akses stack agar tidak out-of-bounds untuk LOD dan ST0, melindungi penulisan ke slot frame header melalui `isProtectedFrameSlot`, memeriksa operasi aritmatika agar tidak menyebabkan overflow atau underflow menggunakan `checkedAdd`, `checkedSub`, `checkedMul`, dan `checkedNeg`, menjamin pembagian dan modulo tidak membagi dengan nol, serta memastikan kedalaman stack tidak melebihi batas maksimum untuk mencegah stack overflow.

3.6 Format Output

Output program Milestone 4 terdiri dari tujuh bagian: `tab`, `btabs`, `atab`, parse tree, decorated AST, intermediate code, dan interpreter output (atau interpreter errors jika terjadi runtime error). Intermediate code ditampilkan dalam format baris demi baris dengan nomor instruksi, opcode, level, dan operand. Interpreter output menampilkan hasil eksekusi program, termasuk nilai yang ditulis oleh `writeln`.

Bab 4

Pengujian

4.1 Strategi Pengujian

Pengujian dilakukan dengan membangun program menggunakan `make -B`, menjalankan `arion_parser` pada input Milestone 4, lalu membandingkan output intermediate code dan interpreter output dengan hasil eksekusi yang diharapkan. Command yang digunakan adalah sebagai berikut.

Command Pengujian

```
make -B
for i in 1 2 3 4 5 6 7; do
    ./arion_parser test/milestone-4/input-m4-$i.txt output-m4-$i.txt
    echo "Test $i exit code: $?"
done
```

Seluruh test case yang digunakan berhasil menghasilkan output intermediate code dan interpreter output yang sesuai dengan ekspektasi. Laporan ini menampilkan sembilan kasus yang mewakili operasi dasar, control flow, perulangan, case statement, array, runtime error, nested procedure, serta kasus yang mengungkapkan keterbatasan implementasi.

Tabel 4.1: Ringkasan Pengujian Milestone 4

No	File Input	Fokus Pengujian
1	input-m4-1.txt	Operasi dasar: assignment, ekspresi aritmatika, div, dan writeln.
2	input-m4-2.txt	Control flow bersarang: if-then-else dengan if di dalam then.
3	input-m4-3.txt	Perulangan while dengan akumulator dan counter.
4	input-m4-4.txt	Perulangan for-to dengan faktorial.
5	input-m4-5.txt	Case statement dengan multiple branch.
6	input-m4-6.txt	Array 2D: assignment dan access dengan ASTO/ALOD.
7	input-m4-7.txt	Runtime error: division by zero detection.

No	File Input	Fokus Pengujian
8	input-m4-8.txt	Nested procedure dengan akses variabel non-lokal melalui static link.
9	input-m4-9.txt	Mutual recursion (keterbatasan: tidak mendukung forward declaration).

4.2 Detail Pengujian

4.2.1 Kasus 1: Operasi Dasar

Input

```
program BasicMath;  
var a, b, c: integer;  
begin  
  a := 10;  
  b := 3;  
  c := (a + b) * (a - b) div 2;  
  writeln(c);  
end.
```

Cuplikan Output

```
=== Intermediate Code ===  
0 INT 0 6  
1 JMP 0 2  
2 LIT 0 10  
3 STO 0 3  
4 LIT 0 3  
5 STO 0 4  
6 LOD 0 3  
7 LOD 0 4  
8 OPR 0 2  
9 LOD 0 3  
10 LOD 0 4  
11 OPR 0 3  
12 OPR 0 4  
13 LIT 0 2  
14 OPR 0 5  
15 STO 0 5  
16 LOD 0 5  
17 OPR 0 14  
18 RET 0 0  
  
=== Interpreter Output ===  
39
```

Kasus ini menunjukkan bahwa operasi dasar assignment, load, store, dan ekspresi aritmatika berhasil diterjemahkan ke intermediate code. Ekspresi $(a + b) * (a - b) \text{ div } 2$ dievaluasi menjadi 39, sesuai dengan hasil perhitungan manual. Hal ini membuktikan bahwa intermediate code generator mampu meratakan ekspresi kompleks menjadi instruksi sekuensial dan interpreter mampu mengeksekusinya dengan benar.

4.2.2 Kasus 2: Control Flow Bersarang

Input

```
program IfElse;
```

```
var x, y: integer;
begin
  x := 5;
  y := 10;
  if x > 0 then
    begin
      if y > 5 then
        writeln(1)
      else
        writeln(2);
      end
    else
      writeln(3);
    end.
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 5
1 JMP 0 2
2 LIT 0 5
3 STO 0 3
4 LIT 0 10
5 STO 0 4
6 LOD 0 3
7 LIT 0 0
8 OPR 0 11
9 JPC 0 22
10 LOD 0 4
11 LIT 0 5
12 OPR 0 11
13 JPC 0 17
14 LIT 0 1
15 OPR 0 14
16 JMP 0 19
17 LIT 0 2
18 OPR 0 14
19 JMP 0 22
20 LIT 0 3
21 OPR 0 14
22 RET 0 0

=== Interpreter Output ===
1
```

Kasus ini menguji nested if-then-else. Kondisi $x > 0$ dan $y > 5$ keduanya benar, sehingga interpreter mencetak 1. Instruksi JMP dan JPC berhasil melompati blok yang tidak perlu dieksekusi, yang membuktikan bahwa translasi control flow bersarang dari decorated AST ke instruksi lompatan telah berhasil dilakukan.

4.2.3 Kasus 3: Perulangan While

Input

```
program WhileLoop;
var i, sum: integer;
begin
  i := 1;
  sum := 0;
  while i <= 5 do
    begin
      sum := sum + i;
      i := i + 1;
    end;
    writeln(sum);
  end.
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 5
1 JMP 0 2
2 LIT 0 1
3 STO 0 3
4 LIT 0 0
5 STO 0 4
6 LOD 0 3
6 LIT 0 5
8 OPR 0 10
9 JPC 0 19
10 LOD 0 4
11 LOD 0 3
12 OPR 0 2
13 STO 0 4
14 LOD 0 3
15 LIT 0 1
16 OPR 0 2
17 STO 0 3
18 JMP 0 6
19 LOD 0 4
20 OPR 0 14
21 RET 0 0

=== Interpreter Output ===
15
```

Kasus ini memvalidasi bahwa `while` loop berhasil diterjemahkan ke intermediate code dengan label dan `jump`. Loop menghitung jumlah $1 + 2 + 3 + 4 + 5 = 15$. Instruksi `JPC` keluar dari loop ketika $i > 5$, dan `JMP` kembali ke evaluasi kondisi, yang membuktikan bahwa mekanisme backpatching untuk perulangan `while` telah bekerja dengan tepat.

4.2.4 Kasus 4: Perulangan For

Input

```
program ForLoop;
var i, fact: integer;
begin
  fact := 1;
  for i := 1 to 5 do
  begin
    fact := fact * i;
  end;
  writeln(fact);
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 5
1 JMP 0 2
2 LIT 0 1
3 STO 0 4
4 LOD 0 4
5 LIT 0 1
6 STO 0 3
7 LOD 0 3
8 LIT 0 5
9 OPR 0 12
10 JPC 0 21
11 LOD 0 4
12 LOD 0 3
```

```
13 OPR 0 4
14 STO 0 4
15 LOD 0 3
16 LIT 0 1
17 OPR 0 2
18 STO 0 3
19 JMP 0 7
20 LOD 0 4
21 OPR 0 14
22 RET 0 0
```

```
=== Interpreter Output ===
120
```

Kasus ini menunjukkan bahwa `for-to` loop berhasil menghitung faktorial 5 yaitu 120. Variabel loop `i` diinisialisasi dengan 1, diincrement setiap iterasi, dan loop berhenti ketika `i > 5`. Hal ini membuktikan bahwa translasi `for` loop dengan inisialisasi, evaluasi batas, body, dan increment telah berhasil dilakukan oleh intermediate code generator dan dieksekusi dengan benar oleh interpreter.

4.2.5 Kasus 5: Case Statement

Input

```
program CaseStmt;
var grade: integer;
begin
  grade := 2;
  case grade of
    1: writeln(10);
    2: writeln(20);
    3: writeln(30);
  end;
end.
```

Cuplikan Output

```
=== Intermediate Code ===
0 INT 0 4
1 JMP 0 2
2 LIT 0 2
3 STO 0 3
4 LIT 0 2
5 LIT 0 1
6 OPR 0 7
7 JPC 0 11
8 LIT 0 10
9 OPR 0 14
10 JMP 0 25
11 LIT 0 2
12 LIT 0 2
13 OPR 0 7
14 JPC 0 18
15 LIT 0 20
16 OPR 0 14
17 JMP 0 25
18 LIT 0 2
19 LIT 0 3
20 OPR 0 7
21 JPC 0 25
22 LIT 0 30
23 OPR 0 14
24 JMP 0 25
25 RET 0 0

=== Interpreter Output ===
```

20

Kasus ini menguji **case** statement dengan tiga branch. Nilai **grade** = 2 cocok dengan branch kedua, sehingga interpreter mencetak 20. Semua **JMP** ke akhir case di-backpatch dengan benar, memastikan eksekusi tidak jatuh ke branch berikutnya, yang membuktikan bahwa translasi case statement dengan backpatching seluruh branch exit telah berhasil dilakukan.

4.2.6 Kasus 6: Array Dua Dimensi

Input

```
program ArrayTwoDRuntime;  
  
var i,j: integer;  
    a: array[1..2] of array[1..3] of integer;  
  
begin  
    i := 2;  
    j := 3;  
    a[i][j] := 42;  
    writeln(a[i][j]);  
end.
```

Cuplikan Output

```
=== Intermediate Code ===  
0 INT 0 11  
1 JMP 0 2  
2 LIT 0 2  
3 STO 0 3  
4 LIT 0 3  
5 STO 0 4  
6 LIT 0 42  
7 LOD 0 3  
8 LOD 0 4  
9 ASTO 0 5:2:2:1:2:1:3  
10 LOD 0 3  
11 LOD 0 4  
12 ALOD 0 5:2:2:1:2:1:3  
13 OPR 0 14  
14 RET 0 0  
  
=== Interpreter Output ===  
42
```

Kasus ini menunjukkan bahwa array dua dimensi dapat diakses melalui instruksi **ASTO** dan **ALOD**. Operand **5:2:2:1:2:1:3** menyimpan metadata array: base offset 5, 2 dimensi, 2 indeks disediakan, batas rendah dan tinggi untuk setiap dimensi (1..2 dan 1..3). Interpreter berhasil menyimpan nilai 42 dan membacanya kembali, yang membuktikan bahwa intermediate code generator mampu menghasilkan instruksi **ASTO** dan **ALOD** serta interpreter mampu mengeksekusi akses array dua dimensi dengan benar.

4.2.7 Kasus 7: Runtime Error Division by Zero

Input

```
program DivZero;  
begin
```

```
writeln(10 div 0);  
end.
```

Cuplikan Output

```
=== Intermediate Code ===  
0 INT 0 3  
1 JMP 0 2  
2 LIT 0 10  
3 LIT 0 0  
4 OPR 0 5  
5 OPR 0 14  
6 RET 0 0  
  
=== Interpreter Output ===  
  
=== Interpreter Errors ===  
- Interpreter: division by zero
```

Kasus ini menguji kemampuan interpreter mendeteksi runtime error. Saat mengeksekusi operasi pembagian `10 div 0`, interpreter menghentikan eksekusi dan melaporkan error informatif tanpa menyebabkan crash, yang membuktikan bahwa mekanisme proteksi runtime pada interpreter telah berhasil mengenali dan menangani kesalahan `division by zero` dengan aman.

4.2.8 Kasus 8: Nested Procedure dengan Static Link

Input

```
program NestedProc;  
procedure outer;  
  var x: integer;  
  procedure inner;  
  begin  
    x := x + 1;  
    writeln(x);  
  end;  
begin  
  x := 10;  
  inner;  
end;  
begin  
  outer;  
end.
```

Cuplikan Output

```
=== Intermediate Code ===  
0 INT 0 3  
1 JMP 0 17  
2 JMP 0 12  
3 JMP 0 4  
4 INT 0 3  
5 LOD 1 3  
6 LIT 0 1  
7 OPR 0 2  
8 STO 1 3  
9 LOD 1 3  
10 OPR 0 14  
11 RET 0 0  
12 INT 0 4  
13 LIT 0 10  
14 STO 0 3  
15 CAL 0 3  
16 RET 0 0
```

```
17 CAL 0 2
18 RET 0 0
```

```
=== Interpreter Output ===
11
```

Kasus ini menguji prosedur bersarang di mana prosedur `inner` didefinisikan di dalam prosedur `outer` dan mengakses variabel `x` milik `outer`. Instruksi `LOD 1 3` dan `STO 1 3` dengan level 1 membuktikan bahwa interpreter berhasil menelusuri static link chain menggunakan fungsi `base(1, b)` untuk mengakses variabel non-lokal pada scope induk. Output 11 sesuai dengan ekspektasi karena `x` diinisialisasi dengan 10, kemudian diincrement oleh `inner` sebelum dicetak.

4.2.9 Kasus 9: Mutual Recursion (Keterbatasan)

Input

```
program MutualRecursion;
procedure A(n: integer);
begin
  if n > 0 then B(n - 1);
end;
procedure B(n: integer);
begin
  if n > 0 then A(n - 1);
  writeln(n);
end;
begin
  A(3);
end.
```

Cuplikan Output

```
=== Semantic Errors ===
- Undefined procedure: B
```

Kasus ini menguji mutual recursion, yaitu pemanggilan silang antara dua prosedur. Program gagal pada tahap semantic analysis dengan pesan **Undefined procedure: B** karena prosedur B belum dideklarasikan saat A mencoba memanggilmnya. Hal ini menunjukkan bahwa compiler Arion saat ini tidak mendukung *forward declaration*, yang diperlukan untuk mutual recursion dalam bahasa Pascal-like. Keterbatasan ini diakui dan direkomendasikan sebagai perbaikan pada pengembangan berikutnya.

4.3 Pengujian Error Runtime

Selain kasus valid, program juga diuji secara manual terhadap input yang mengandung kesalahan runtime. Contoh kesalahan yang berhasil dideteksi adalah stack overflow pada infinite recursion, array index out of bounds, invalid jump targets, dan integer overflow.

```
Input (stack_overflow.txt):
program StackOverflow;
procedure recur; begin recur; end;
begin recur; end.
```

```
Output:
=== Interpreter Errors ===
- Interpreter: stack overflow
```



```
Input (array_oob_runtime.txt):  
program ArrayOOBRuntime;  
var i: integer; a: array[1..3] of integer;  
begin i := 4; a[i] := 10; writeln(a[i]); end.
```

```
Output:  
=== Interpreter Errors ===  
- Interpreter: IndexOutOfBoundsException
```

4.4 Analisis Kegagalan dan Perbaikan

Selama pengembangan Milestone 4, ditemukan beberapa bug yang signifikan pada intermediate code generator dan interpreter. Berikut adalah analisis kegagalan dan langkah perbaikan yang dilakukan.

Bug 1: Infinite loop pada for statement. Saat pertama kali diimplementasikan, for loop menghasilkan infinite loop karena optimasi INT 0 1 untuk slot sementara batas akhir menyebabkan korupsi stack. Slot sementara ditulis tanpa alokasi memori yang cukup, sehingga nilai batas akhir menimpa data lain di stack. Perbaikan yang dilakukan adalah menghapus optimasi tersebut dan mengembalikan evaluasi batas akhir pada setiap iterasi, yang memastikan stack tetap konsisten.

Bug 2: Case statement backpatch leak. Pada awalnya, hanya branch pertama pada case statement yang di-backpatch untuk melompat ke akhir case. Branch-branch berikutnya tidak di-backpatch, sehingga eksekusi jatuh ke branch berikutnya (*fall-through*) secara tidak sengaja dan menyebabkan infinite loop. Perbaikan yang dilakukan adalah mengubah variabel `endJump` dari tipe integer tunggal menjadi `std::vector<int> endJumps`, sehingga setiap branch memiliki backpatch list sendiri yang semuanya diisi dengan alamat akhir case.

Bug 3: Negative INT pada procedure call. Instruksi INT dengan nilai negatif digunakan untuk membersihkan argumen dari stack caller setelah CAL. Interpreter awalnya menolak nilai negatif dengan pesan `invalid stack size request`. Perbaikan yang dilakukan adalah menambahkan penanganan nilai negatif pada INT, yang menyusutkan stack dengan menghapus elemen dari belakang sesuai nilai absolutnya.

4.5 Analisis Keterbatasan Pengujian

Sembilan test case yang disusun telah mencakup fitur utama intermediate code generation dan interpreter, termasuk nested procedure dengan static link traversal yang membuktikan kebenaran fungsi `base(1, b)`. Namun masih terdapat skenario edge case yang belum diuji secara otomatis, antara lain mutual recursion yang memerlukan forward declaration, operasi modulo dengan variasi operand, perbandingan selain `>` dan `<=`, serta tipe data selain integer (misalnya real, char, boolean) pada level intermediate code. Kasus-kasus tersebut diakui sebagai keterbatasan pengujian saat ini dan direkomendasikan sebagai perluasan pengujian pada pengembangan berikutnya.

Bab 5

Penutup

5.1 Kesimpulan

Milestone 4 berhasil menambahkan tahap intermediate code generation dan interpreter pada Arion Compiler. Program dapat menerima source code, token stream, atau parse tree, kemudian menghasilkan parse tree, AST, symbol table, decorated AST, intermediate code, dan output eksekusi program. Implementasi intermediate code generator telah mengubah decorated AST menjadi daftar instruksi sekuensial yang terdiri dari LIT, LOD, STO, INT, JMP, JPC, OPR, CAL, dan RET. Instruksi-instruksi tersebut kemudian dieksekusi oleh interpreter yang berperan sebagai virtual machine dengan siklus fetch-decode-execute.

Stack machine telah diimplementasikan untuk mengelola memori eksekusi melalui stack frame yang menyimpan variabel lokal, argumen, static link, dynamic link, dan return address. Interpreter mampu menangani translasi control flow dari `if`, `while`, `for`, `repeat`, dan `case` ke dalam instruksi sekuensial dengan label dan jump. Program juga telah mendukung array access dengan satu dan dua dimensi melalui instruksi ALOD dan ASTO.

Berdasarkan pengujian, program berhasil menangani kasus dasar, control flow bersarang, perulangan, array, dan runtime error. Program juga mampu melaporkan beberapa runtime error secara informatif tanpa crash, termasuk division by zero, stack overflow, array out of bounds, invalid jump targets, dan integer overflow.

5.2 Saran

Pengembangan berikutnya dapat menambahkan dukungan untuk function call dalam ekspresi (`FuncCallNode`) pada level intermediate code generation, sehingga nilai return dari fungsi dapat digunakan dalam assignment atau ekspresi lainnya. Selain itu, record field access dapat diperluas agar field dapat diakses dan dimodifikasi pada saat runtime. Boolean operator `and`, `or`, dan `not` juga dapat ditambahkan ke instruction set agar kondisi kompleks dapat dieksekusi.

Untuk meningkatkan keamanan, interpreter dapat ditambahkan dengan deteksi use-after-free pada variabel yang sudah di-pop dari stack, serta validasi tipe runtime untuk memastikan operasi hanya dilakukan pada tipe data yang kompatibel. Pengujian juga dapat diperluas dengan kasus edge case yang lebih banyak, termasuk



nested procedure, array multidimensi lebih dari dua, dan kombinasi control flow yang lebih kompleks.

Bab 6

Lampiran

6.1 Tautan GitHub

Repository proyek dapat diakses melalui tautan <https://github.com/Vixrlie/KAI-Tubes-IF2224-2026>. Halaman release yang berisi versi final dan catatan rilis tersedia di <https://github.com/Vixrlie/KAI-Tubes-IF2224-2026/releases>.

6.2 Tabel Kontribusi

No	Nama Lengkap	NIM	Deskripsi Pekerjaan	Kontribusi
1	Bryan Pratama Putra Hendra	13524067	Revisi milestone sebelumnya dan Intermediate Code Generator (flattening Decorated AST ke TAC, instruksi dasar LIT/LOD/STO/INT).	25%
2	Vincent Rionarlie	13524031	Interpreter dan memori eksekusi (VM stack machine, stack frame dengan SL/DL/RA, siklus Fetch-Decode-Execute).	25%
3	Muhammad Aufar Rizqi Kusuma	13524061	Control flow, operasi, dan keamanan mesin (translasi IF/WHILE ke JMP/JPC, operasi OPR, proteksi stack overflow dan numerical overflow).	25%
4	Jonathan Kris Wicaksono	13524023	Pengujian, laporan, dan rilis (penyusunan test case, penulisan laporan PDF, manajemen release GitHub beserta README).	25%

6.3 Cara Menjalankan Program

```
make -B
make run INPUT=test/milestone-4/input-m4-1.txt
make run-output INPUT=test/milestone-4/input-m4-1.txt OUTPUT=output-m4.txt
```

Bibliografi

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. 2nd edition. Addison-Wesley, 2007.
- [2] Andrew W. Appel. *Modern Compiler Implementation in C*. 2nd edition. Cambridge University Press, 1998.
- [3] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. 2nd edition. Morgan Kaufmann, 2011.
- [4] Kenneth C. Louden. *Compiler Construction: Principles and Practice*. PWS Publishing Company, 1997.
- [5] Terence Parr. *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Pragmatic Bookshelf, 2010.
- [6] Niklaus Wirth. *Compiler Construction*. Addison-Wesley, 1996.